AL2002 Artificial Intelligence Lab Manual
ver 2.0

Saad Ahmad, *BS*
Inst., Dept. of Computer Science
National University of Computer & Emerging Sciences

Artificial Intelligence Lab Manual

Saad Ahmad
Instructor, Dept. of Computer Science
National University of Computer & Emerging Sciences
160 Industrial Estate, Peshawar, KPK, Pakistan

Version History
1.0 Spring 2026: Compilation of all previous labs in book format with minor updates & corrections {**Saad Ahmad**}
2.0 Spring 2026: Restructured manual: updated chapter titles, added Simulated Annealing, new CSP chapter (Lab 9), Linear & Logistic Regression in Ch. 13, improved heuristics explanation, added TikZ figures & code snippets {**Saad Ahmad**}

This manual is customized for course AL2002 Artificial Intelligence taught at the National University of Computer & Emerging Sciences, Peshawar, Pakistan. The lab portion of this course provides cover for 1 credit hour. Evaluation for this lab is at discretion of your instructor and/or teacher.

The manual can also be used as a learning tool for other artificial intelligence courses both at undergraduate or graduate level. Each chapter in this manual is intended to serve as a separate lab, although instructors should note that some chapter's require more than one lab sessions to complete.

Contents

I MACHINE LEARNING **5**

1 Supervised Learning: K-Nearest Neighbors (KNN) **7**

1.1 What is Machine Learning? . . . 7
1.2 Supervised Learning . . . 7
1.3 What is KNN? . . . 7
1.3.1 Key Points . . . 8
1.4 The Problem: Paper Tissue Classification . . . 8
1.5 Example 1: Building KNN from Scratch . . . 9
1.5.1 Step 1: Represent the Data . . . 9
1.5.2 Step 2: Calculate Euclidean Distance . . . 9
1.5.3 Step 3: Find K Nearest Neighbors . . . 10
1.5.4 Step 4: Vote and Classify . . . 10
1.6 Complete KNN Program . . . 11
1.7 Experiment & Explore . . . 12
1.7.1 Experiment 1: Effect of k . . . 12
1.7.2 Experiment 2: More Training Data . . . 12
1.7.3 Experiment 3: Classify Multiple Points . . . 13
1.8 Task 1: Scratch KNN on the Iris Dataset . . . 13
1.9 Example 2: Using Scikit-Learn for KNN . . . 14
1.10 Task 2: Using Scikit-Learn for Real-World Data . . . 14
1.10.1 Program Requirements . . . 15
1.10.2 Skeleton Code . . . 15
1.11 Reflection . . . 16

Part I

MACHINE LEARNING

Chapter 1

Supervised Learning: K-Nearest Neighbors (KNN)

1.1 What is Machine Learning?

Machine Learning (ML) is a field of Artificial Intelligence (AI) that focused on building systems that can learn from and make decisions based on data. Instead of being explicitly programmed with every rule, an ML algorithm uses patterns in data to improve its performance on a specific task over time.

> "A computer program is said to learn from experience E with respect to some task T and some performance measure P, if its performance on T, as measured by P, improves with experience E."
> – Tom Mitchell

1.2 Supervised Learning

Supervised Learning is a type of machine learning where the algorithm is trained on a **labeled dataset**. This means that for every piece of data in the training set, we already know the correct answer (the label).

The goal is for the algorithm to learn the mapping from inputs to outputs so that it can accurately predict the labels for new, unseen data. Common tasks in supervised learning include:

- **Classification:** Predicting a category (e.g., "Good" or "Bad", "Spam" or "Not Spam").
- **Regression:** Predicting a continuous numerical value (e.g., house prices).

1.3 What is KNN?

K-Nearest Neighbors (KNN) is one of the simplest machine learning algorithms. The idea is:

> To classify an unknown data point, look at the k closest known data points and pick the label that appears most often among them.

Example: Imagine you move to a new city and want to know if a neighborhood is "safe" or "unsafe." You ask the 5 people living closest to you. If 4 say "safe" and 1 says "unsafe," you classify the neighborhood as "safe." That's KNN with $k = 5$.

1.3.1 Key Points

- KNN is a **supervised** algorithm — it needs labeled training data.
- It does **no training** — it simply stores the data and computes distances at prediction time ("lazy learner").
- The parameter k controls how many neighbors to consider. k should be an **odd number** to avoid ties.
- It uses a **distance function** (usually Euclidean distance) to find the nearest neighbors.

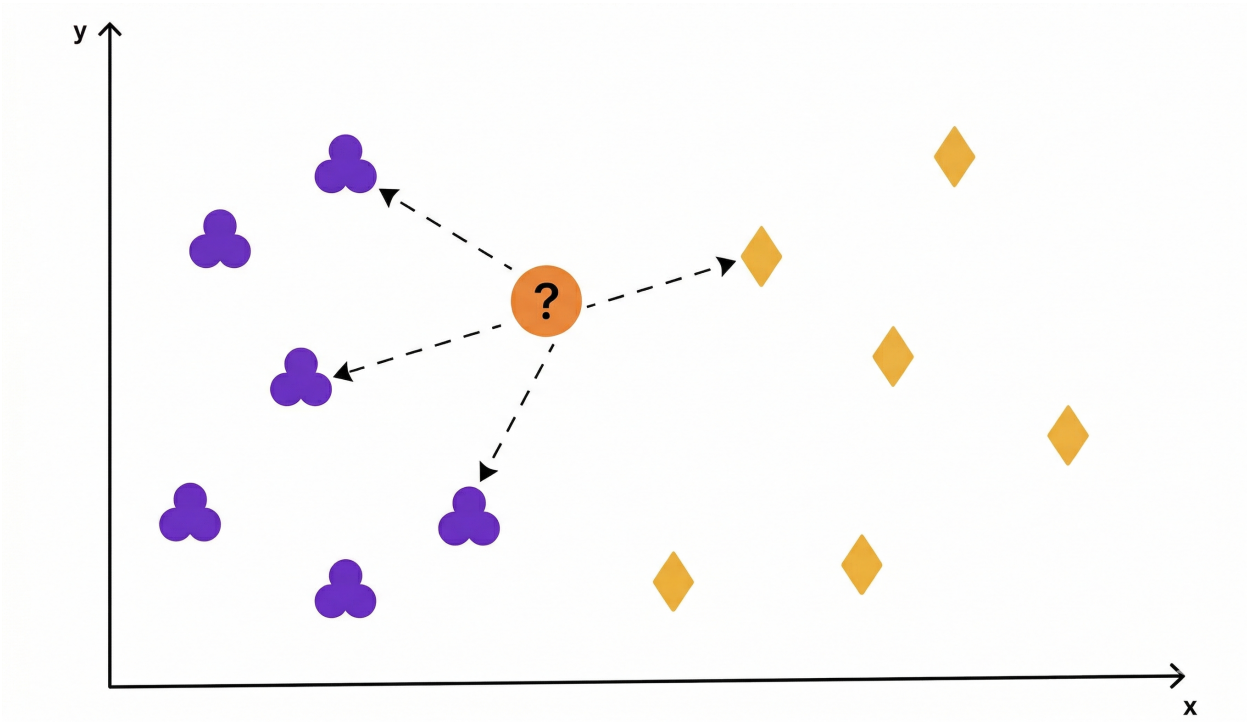

Figure 1.1: KNN classifies the unknown point (star) based on its k nearest neighbors.

1.4 The Problem: Paper Tissue Classification

A factory produces paper tissues and tests them on two properties:

- X_1 = Acid Durability (how well it resists acid)
- X_2 = Strength

Based on surveys, each tissue is classified as "Good" or "Bad." Here is our training data:

Acid Durability (X_1)	**Strength (X_2)**	**Label**
7	7	Bad
7	4	Bad
3	4	Good
1	4	Good

Question: A new tissue has $X_1 = 3$ and $X_2 = 7$. Is it Good or Bad?
We will build a KNN classifier to answer this.

1.5 Example 1: Building KNN from Scratch

To fully understand how KNN works, we will first implement it piece by piece without using any machine learning libraries.

1.5.1 Step 1: Represent the Data

We store training data as a list of lists. Each item has $[X_1, X_2, \text{label}]$.

```python
# Each row: [acid_durability, strength, label]
training_data = [
    [7, 7, "Bad"],
    [7, 4, "Bad"],
    [3, 4, "Good"],
    [1, 4, "Good"]
]

# The new point we want to classify
new_point = [3, 7]
```

Listing 1.1: Training data

Checkpoint 1:

```python
print(training_data[0])          # [7, 7, 'Bad']
print(training_data[0][2])       # 'Bad' (the label)
print(new_point)                 # [3, 7]
```

1.5.2 Step 2: Calculate Euclidean Distance

The Euclidean distance between two points (x_1, y_1) and (x_2, y_2) is:

$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

We can skip the square root for comparison purposes (since $\sqrt{a} < \sqrt{b}$ if and only if $a < b$), but we'll include it for clarity.

```python
import math

def euclidean_distance(point1, point2):
    # point1 and point2 are lists like [x1, x2]
    total = 0
    for i in range(len(point1)):
        diff = point1[i] - point2[i]
        total = total + (diff * diff)
    distance = math.sqrt(total)
    return distance
```

Listing 1.2: Euclidean distance function

Checkpoint 2: Verify against the manual calculations.

```python
print(euclidean_distance([3,7], [7,7]))      # 4.0
print(euclidean_distance([3,7], [7,4]))      # 5.0
print(euclidean_distance([3,7], [3,4]))      # 3.0
print(euclidean_distance([3,7], [1,4]))      # 3.605...
```

Question: Which training point is closest to $[3, 7]$?

1.5.3 Step 3: Find K Nearest Neighbors

Now we calculate the distance from the new point to every training point, sort them, and pick the k closest ones.

```python
def get_neighbors(training_data, new_point, k):
    # Step 1: Calculate distance to every training point
    distances = []
    for row in training_data:
        # The features are all columns except the last (label)
        features = [row[0], row[1]]
        label = row[2]
        dist = euclidean_distance(new_point, features)
        distances.append([dist, label])

    # Step 2: Sort by distance (smallest first)
    # Simple bubble sort
    for i in range(len(distances)):
        for j in range(i + 1, len(distances)):
            if distances[j][0] < distances[i][0]:
                # Swap
                temp = distances[i]
                distances[i] = distances[j]
                distances[j] = temp

    # Step 3: Pick the first k entries
    neighbors = []
    for i in range(k):
        neighbors.append(distances[i])

    return neighbors
```

Listing 1.3: Finding the K nearest neighbors

Checkpoint 3: Test with $k = 3$.

```
neighbors = get_neighbors(training_data, [3,7], 3)
for n in neighbors:
print("Distance:", round(n[0], 2), "Label:", n[1])
```

Expected output (approximately):

```
Distance: 3.0  Label: Good
Distance: 3.61 Label: Good
Distance: 4.0  Label: Bad
```

1.5.4 Step 4: Vote and Classify

Count how many of the k neighbors belong to each class. The class with the most votes wins.

```python
def classify(training_data, new_point, k):
    neighbors = get_neighbors(training_data, new_point, k)

    # Count votes for each label
    votes = {}
    for neighbor in neighbors:
        label = neighbor[1]
        if label in votes:
            votes[label] = votes[label] + 1
        else:
            votes[label] = 1
```

```python

    # Find the label with the most votes
    best_label = None
    best_count = 0
    for label in votes:
        if votes[label] > best_count:
            best_count = votes[label]
            best_label = label

    return best_label
```

Listing 1.4: Majority voting to classify

Checkpoint 4: Classify the new tissue.

```python
result = classify(training_data, [3, 7], 3)
print("Classification:", result)                    # Expected: 'Good'
```

We had 2 Good neighbors and 1 Bad neighbor, so majority vote = Good.

1.6 Complete KNN Program

Here is the complete program. **Type it out yourself** to reinforce understanding.

```python
import math

# ---- Training Data ----
training_data = [
    [7, 7, "Bad"],
    [7, 4, "Bad"],
    [3, 4, "Good"],
    [1, 4, "Good"]
]

# ---- Functions ----
def euclidean_distance(point1, point2):
    total = 0
    for i in range(len(point1)):
        diff = point1[i] - point2[i]
        total = total + (diff * diff)
    return math.sqrt(total)

def get_neighbors(training_data, new_point, k):
    distances = []
    for row in training_data:
        features = [row[0], row[1]]
        label = row[2]
        dist = euclidean_distance(new_point, features)
        distances.append([dist, label])

    # Sort by distance (bubble sort)
    for i in range(len(distances)):
        for j in range(i + 1, len(distances)):
            if distances[j][0] < distances[i][0]:
                temp = distances[i]
                distances[i] = distances[j]
                distances[j] = temp

    neighbors = []
    for i in range(k):
        neighbors.append(distances[i])
```

```python
    return neighbors

def classify(training_data, new_point, k):
    neighbors = get_neighbors(training_data, new_point, k)

    votes = {}
    for neighbor in neighbors:
        label = neighbor[1]
        if label in votes:
            votes[label] = votes[label] + 1
        else:
            votes[label] = 1

    best_label = None
    best_count = 0
    for label in votes:
        if votes[label] > best_count:
            best_count = votes[label]
            best_label = label
    return best_label

# ---- Classify a new point ----
new_point = [3, 7]
k = 3
result = classify(training_data, new_point, k)
print("New point:", new_point)
print("Using k =", k)
print("Classification:", result)
```

Listing 1.5: Complete KNN classifier

1.7 Experiment & Explore

1.7.1 Experiment 1: Effect of k

Using the same training data, classify the point $[3, 7]$ with different values of k and record the results:

k	**Classification**	**Neighbor Labels (in order)**
1		
3		

Question: Does the classification change when you use $k = 1$ vs $k = 3$? Why or why not?

1.7.2 Experiment 2: More Training Data

Add these new data points to `training_data` and re-run:

X_1	X_2	**Label**
3	6	Good
5	5	Bad
2	8	Good
8	3	Bad

Classify $[3, 7]$ again with $k = 3$ and $k = 5$. Did the results change? Why?

1.7.3 Experiment 3: Classify Multiple Points

Write a loop that classifies the following test points and prints the result for each:

X_1	X_2	**Your KNN Prediction** ($k = 3$)
5	6	
2	3	
8	8	
4	5	

Hint:

```
test_points = [[5,6], [2,3], [8,8], [4,5]]
for point in test_points:
result = classify(training_data, point, 3)
print("Point:", point, "->", result)
```

1.8 Task 1: Scratch KNN on the Iris Dataset

Apply your KNN to a bigger dataset. **The Iris dataset** contains 3 types of flowers, each described by 4 measurements: sepal length, sepal width, petal length, and petal width.

Complete the TODO functions below.

```python
import math

# Iris training data (simplified: 12 samples)
# [sepal_length, sepal_width, petal_length, petal_width, species]
training_data = [
    [5.1, 3.5, 1.4, 0.2, "Setosa"],
    [4.9, 3.0, 1.4, 0.2, "Setosa"],
    [5.0, 3.4, 1.5, 0.2, "Setosa"],
    [4.8, 3.1, 1.6, 0.2, "Setosa"],
    [7.0, 3.2, 4.7, 1.4, "Versicolor"],
    [6.4, 3.2, 4.5, 1.5, "Versicolor"],
    [5.5, 2.3, 4.0, 1.3, "Versicolor"],
    [6.5, 2.8, 4.6, 1.5, "Versicolor"],
    [6.3, 3.3, 6.0, 2.5, "Virginica"],
    [7.1, 3.0, 5.9, 2.1, "Virginica"],
    [6.5, 3.0, 5.8, 1.8, "Virginica"],
    [7.6, 3.0, 6.6, 2.1, "Virginica"]
]

def euclidean_distance(point1, point2):
    """TODO: Calculate Euclidean distance.
       This time, points have 4 features instead of 2.
       Your earlier function should already work!"""
    pass

def classify(training_data, new_point, k):
    """TODO: Find k nearest neighbors and return the
       majority label. Reuse your earlier approach."""
    pass

# ---- Test with these flowers ----
test_flowers = [
    [5.0, 3.3, 1.4, 0.2],   # What species?
    [6.0, 2.7, 4.5, 1.5],   # What species?
    [7.2, 3.2, 6.0, 1.8],   # What species?
```

```
]

for flower in test_flowers:
    result = classify(training_data, flower, 3)
    print("Flower:", flower, "-> Species:", result)
```

Listing 1.6: KNN on the Iris dataset (complete the TODOs)

Tasks:

1. Complete the two `TODO` functions and run the program.
2. Note: your `euclidean_distance` function already handles any number of features — verify this by checking `len(point1)` inside the function.
3. **Bonus:** Try $k = 1$, $k = 3$, $k = 5$. Do the results change for any test flower?

1.9 Example 2: Using Scikit-Learn for KNN

Building an algorithm from scratch is great for learning, but in the industry, we use optimized libraries like **Scikit-Learn** and **Pandas**. Here is an example of how to use them to load data, split it, and train a classifier.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# 1. Load data (assuming iris.csv is in your directory)
df = pd.read_csv('iris.csv')

# 2. Prepare data: Separate features (X) and target label (y)
X = df.drop('species', axis=1)
y = df['species']

# 3. Split data into training (80%) and testing (20%)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
    random_state=42)

# 4. Create and train the model (K=3)
knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train, y_train)

# 5. Predict and evaluate
y_pred = knn.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
```

Listing 1.7: Scikit-Learn KNN Example

1.10 Task 2: Using Scikit-Learn for Real-World Data

Now it's your turn. In this task, you will use the `fruit_data_with_colors.csv` dataset and **Matplotlib** for visualization. This dataset contains measurements for different types of fruits (apples, mandarin, orange, lemon).

1.10.1 Program Requirements

Complete the following steps in a new Python script:

1. **Load Data:** Use `pandas.read_csv()` to load the dataset.
2. **Clean Data:**
   - Remove any columns that contain text or categorical values (like fruit names).
   - Handle missing values by replacing them with the **mean** of their respective column.
3. **Train-Test Split:** Divide the data. Use the first 50 rows for **training** and the remaining 10 rows for **testing**.
4. **Model Application:** Apply the `KNeighborsClassifier` from `sklearn.neighbors`.
5. **Hyperparameter Tuning:** Run the model for every value of K from **1 to 10**.
6. **Visualization:** Use `matplotlib` to plot K (x-axis) vs. Accuracy Score (y-axis).

1.10.2 Skeleton Code

```python
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# 1. Load the data
df = pd.read_csv('fruit_data_with_colors.csv')

# 2. Prepare data
# TODO: Drop categorical columns
# TODO: Fill missing values with mean
df = df.select_dtypes(exclude=['object'])
df = df.fillna(df.mean())

# 3. Split data (First 50 for training, rest for testing)
train_df = df.iloc[:50]
test_df = df.iloc[50:]

X_train = train_df.drop('fruit_label', axis=1)
y_train = train_df['fruit_label']
X_test = test_df.drop('fruit_label', axis=1)
y_test = test_df['fruit_label']

# 4 & 5. Loop through K values
k_values = range(1, 11)
scores = []

for k in k_values:
    # TODO: Create KNN classifier
    # TODO: Fit the model
    # TODO: Predict and calculate accuracy
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train, y_train)
    y_pred = knn.predict(X_test)
```

```
    scores.append(accuracy_score(y_test, y_pred))

# 6. Plotting
plt.plot(k_values, scores, marker='o')
plt.xlabel('Value of K')
plt.ylabel('Accuracy Score')
plt.title('K vs Accuracy on Fruit Dataset')
plt.show()
```

Listing 1.8: Task 2 Skeleton

Questions for Task 2:

- Which value of K gave the highest accuracy?
- Why did we drop the text columns before training?
- What would happen if we used the entire dataset for both training and testing?

1.11 Reflection

Answer the following questions in your own words (2–3 sentences each):

1. Why is k usually chosen as an odd number?
2. What happens if k is too small (e.g., $k = 1$)? What if k is too large (e.g., k equals the total number of training points)?
3. Why is KNN called a "lazy learner"? Is this an advantage or a disadvantage?
4. If one feature (e.g., Acid Durability) ranges from 0 to 100 and another (e.g., Strength) ranges from 0 to 10, how would this affect KNN? What could you do to fix it?

Bibliography

[1] Agents in artificial intelligence.

[2] Artificial intelligence.

[3] Artificial intelligence.

[4] Artificial intelligence - agents and environments.

[5] Cs6-1 lab 06.

[6] Networkx - python software package for complex networks.

[7] Types of artificial intelligence.

[8] What is intelligent agent in ai - types and function.